Practical No.1 Lab Manual: Employee Management System

Object-Oriented Programming Practical — Constructors in Java

---

1. Title

Employee Management System — Demonstrating Default and Parameterized Constructors

---

2. Objective

- To understand the concept of **constructors** in Java (Object-Oriented Programming).
- To differentiate between a **default constructor** and a **parameterized constructor**.
- To create a class representing company/employee data and instantiate objects using both constructor types.
- To display employee details using object methods.

---

3. Scenario

A company wants to store and manage employee details such as Employee ID, Name, Designation, and Salary. As a programmer, you are required to design a class called `CompanyData` that models this information. Two employee records must be created:

1. One employee object created using a **default constructor** (with pre-set/default values).
2. Another employee object created using a **parameterized constructor** (with custom values supplied at the time of object creation).

Both objects should be able to display their respective employee details.

---

4. Pre-requisites

- Basic knowledge of Java syntax.

- Understanding of classes, objects, and methods.
- JDK installed (version 8 or above).
- A text editor or IDE (Eclipse, IntelliJ IDEA, VS Code, or a simple terminal with `javac`/`java`).

---

5. Theory

5.1 What is a Constructor?

A constructor is a special method in Java that is automatically invoked when an object of a class is created. It has the same name as the class and no return type (not even `void`). Its main purpose is to **initialize the object's state** (its instance variables).

5.2 Types of Constructors

Constructor Type	Description
Default Constructor	A constructor with no arguments. If a class has no constructor defined at all, Java automatically provides an implicit default constructor. If explicitly written, it can initialize instance variables with fixed/default values.
Parameterized Constructor	A constructor that accepts arguments, allowing the object to be initialized with specific values passed by the caller at the time of creation.

5.3 Constructor Overloading

Java allows a class to have multiple constructors with different parameter lists — this is called **constructor overloading**. In this lab, the `CompanyData` class will have two constructors: one with no parameters (default) and one with parameters (parameterized).

5.4 Why This Matters in Real Systems

In an Employee Management System, a default constructor may be used to create a placeholder or "blank" employee record (e.g., before details are filled in via a form), while a parameterized constructor is used when complete employee data is already available and needs to be stored immediately, such as during employee onboarding.

---

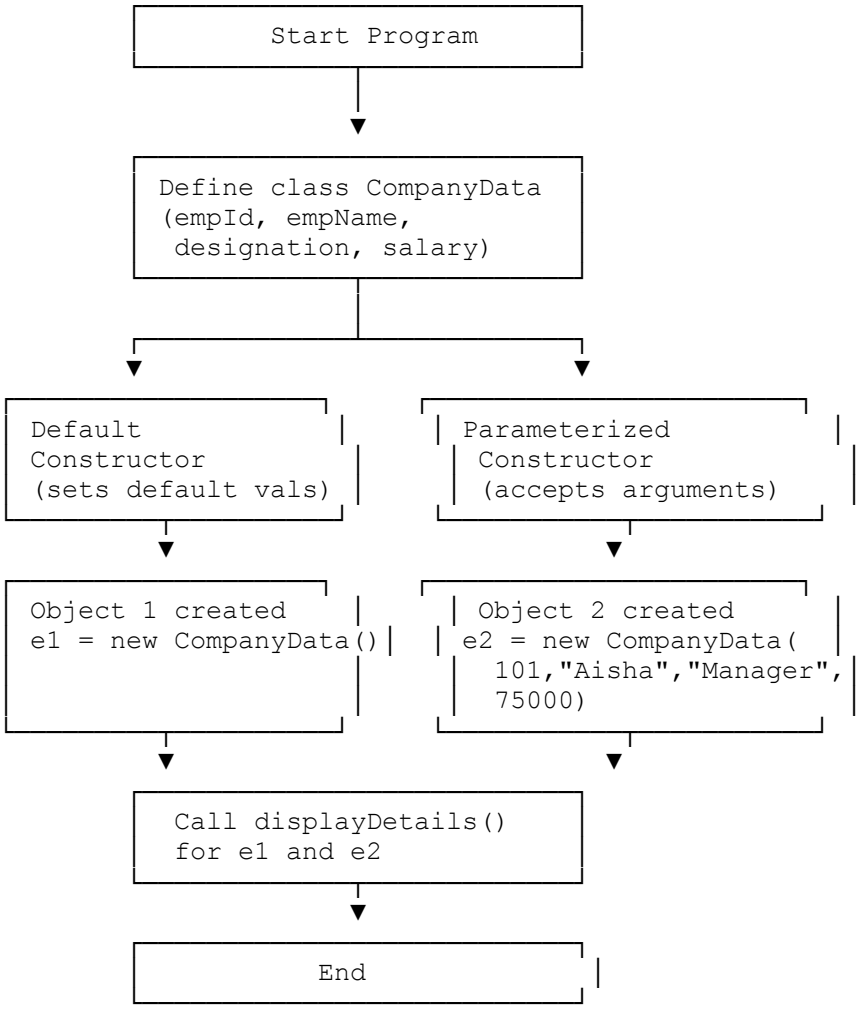
6. Algorithm

1. Start.
2. Define a class `CompanyData` with instance variables: `empId`, `empName`, `designation`, and `salary`.

3. Define a **default constructor** that initializes these variables with default/dummy values.
4. Define a **parameterized constructor** that accepts values for these variables as arguments and assigns them to the instance variables.
5. Define a method `displayDetails()` to print employee details.
6. In the `main` method: a. Create Object 1 using the default constructor. b. Create Object 2 using the parameterized constructor, passing real employee data.
7. Call `displayDetails()` on both objects.
8. Stop.

---

7. Flow of Execution (Diagram)



---

8. Source Code

9. Steps to Compile and Run

1. Save the file as `EmployeeManagementSystem.java` (the file name must match the public class name).
2. Open a terminal / command prompt in the folder containing the file.
3. Compile the program:
4. `javac EmployeeManagementSystem.java`
5. Run the compiled program:
6. `java EmployeeManagementSystem`

10. Expected Output

```
Employee Details (Default Constructor):
-----------------------------------
Employee ID   : 0
Employee Name : Not Assigned
Designation   : Trainee
Salary        : Rs. 15000.0
-----------------------------------

Employee Details (Parameterized Constructor):
-----------------------------------
Employee ID   : 101
Employee Name : Aisha Sharma
Designation   : Manager
Salary        : Rs. 75000.0
-----------------------------------
```

11. Observation

- The object `emp1`, created using the **default constructor**, automatically received fixed/placeholder values (`empId = 0`, `empName = "Not Assigned"`, etc.) without the programmer supplying any data at creation time.
- The object `emp2`, created using the **parameterized constructor**, received custom values (`101`, `"Aisha Sharma"`, `"Manager"`, `75000.00`) directly at the time of object creation.
- Both objects belong to the same class (`CompanyData`) but hold independent data because each object has its own copy of instance variables.
- This demonstrates **constructor overloading**, where multiple constructors coexist in a class with different parameter lists, and Java determines which one to invoke based on the arguments passed during object creation.

12. Viva / Quiz Questions

1. What is a constructor in Java, and how is it different from a normal method?
2. What happens if a class does not define any constructor at all?
3. Can a class have more than one constructor? What is this feature called?
4. What is the purpose of the `this` keyword in the parameterized constructor?
5. Can a constructor have a return type? Justify your answer.
6. What would happen if you tried to call `new CompanyData(101, "Aisha")` with this class? Why?
7. Differentiate between constructor overloading and method overloading.

---

13. Precautions

- Ensure the file name exactly matches the public class name (`EmployeeManagementSystem.java`).
- Do not give a constructor a return type — doing so turns it into a regular method, not a constructor.
- Ensure the number and type of arguments passed while creating an object match one of the defined constructors exactly.
- Use the `this` keyword carefully to distinguish between instance variables and constructor parameters when their names are identical.

---

14. Conclusion

In this lab, a `CompanyData` class was designed to model employee details for a company. Two objects were successfully created — one using a default constructor that initialized placeholder values, and another using a parameterized constructor that initialized real employee data supplied at runtime. This exercise reinforced the concept of constructors, constructor overloading, and how object initialization works in Java-based Object-Oriented Programming.

---

15. Practice Exercises (For Students)

1. Add a third constructor that accepts only `empId` and `empName`, setting `designation` and `salary` to defaults.
2. Add a method `giveRaise(double amount)` that increases an employee's salary, and test it on both objects.
3. Modify the program to store multiple employees in an array or `ArrayList` and display all their details using a loop.

4. Add input validation to ensure `salary` is never set to a negative value.